

# Progetto Architetture Dati

**Confronto scalabilità tra DBMS relazionale e non relazionale**

Dominio HR/retributivo multitenant

**Fabio Compagnoni** — matricola 944954

Università degli Studi di Milano-Bicocca

# Il progetto e l'obiettivo

---

Da un **caso reale** (buste paga in formato XML) si sono progettati **due schemi sui medesimi dati**:

- ▶ un database **relazionale** distribuito — PostgreSQL + Citus;
- ▶ un database **non relazionale** distribuito — MongoDB.

## La domanda

A parità di dominio e di dati, **quale dei due modelli scala meglio e ha performance migliori?**

Due filosofie opposte, stessa classe CAP (entrambi **CP**): normalizzazione e *join* da un lato, aggregato ed *embedding* dall'altro.

## Il dominio: HR e retribuzioni

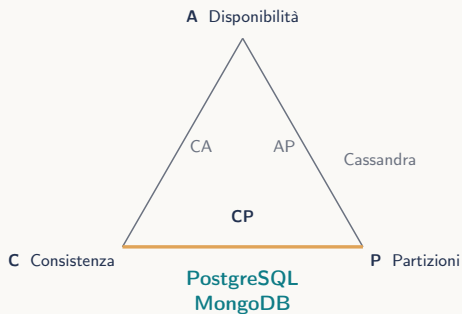
---

- ▶ Gestione del personale e delle **paghe**: **più aziende clienti** sullo stesso sistema (regime **multitenant**).
- ▶ Origine **reale** (XML di cedolini); per il GDPR il dataset è **sintetico ma calibrato**, riproducibile (seed). Base: 30 ditte / 554 dip. → migliaia di ditte.
- ▶ Il **cedolino** è l'aggregato centrale, ma poggia su una struttura relazionale profonda (anagrafiche, cataloghi condivisi).

Perché il `tenant_id` (la ditta) è centrale

È la **chiave di partizionamento**: **distribuisce i dati sui nodi** in **entrambi** i sistemi, tenendo **co-locati** i dati di un'azienda su un solo nodo (una ditta = un nodo). **Ma è anche un limite**: se quel nodo cade, **tutte le query di quell'azienda falliscono**, come confermato dai test sui guasti.

# Il teorema CAP: perché due sistemi CP



Di fronte a un **partizionamento di rete**, un sistema distribuito può garantire solo **2 proprietà su 3**: Consistenza, Disponibilità, Partition tolerance.

Le partizioni **accadono**: la scelta reale è tra **CP** e **AP**. In un dominio **retributivo** la **consistenza è irrinunciabile** (nessun cedolino incoerente), quindi entrambi i sistemi scelti sono **CP**.

**Cassandra**, ad esempio, è un sistema **AP**: in caso di partizione privilegia la disponibilità a scapito della consistenza, quindi non è adatta a un dominio retributivo.

# La scelta dei due DBMS

---

Entrambi **CP**, ma scelti per ragioni distinte, uno per ciascun modello di dato:

## Relazionale: PostgreSQL + Citus

PostgreSQL non è distribuito di default: lo si è reso tale con l'estensione **Citus** (coordinator e worker), che espone primitive di distribuzione **esplicite** (tabelle distribuite, reference table, co-localazione) mantenendo SQL completo, transazioni distribuite (2PC) e integrità referenziale.

## NoSQL: perché proprio MongoDB

Tra le **molte famiglie NoSQL** (chiave-valore, wide-column, grafo, documentale) è il **dominio** a decidere: il cedolino è un **aggregato**, quindi modello **documentale**. Tra i documentali, MongoDB unisce maturità e aggregation framework a **transazioni multi-documento ACID stabili**, non scontate nel mondo NoSQL e richieste dal progetto.



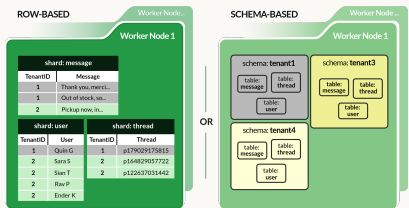
# Schema documentale: la collezione cedolini

Nel modello documentale il cedolino è **un unico documento**: l'anagrafica è uno *snapshot* embedded e voci, ratei, contributi e addizionali sono **array annidati**. Si legge e si scrive con una sola operazione, senza join.

```
db.createCollection("cedolini", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["tenant_id", "cedolino_id", "dipendente_id", "anno", "mese",
              "dipendente", "totali", "voci"],
    properties: {
      tenant_id: {bsonType: "long"}, cedolino_id: {bsonType: "long"},
      dipendente_id: {bsonType: "long"}, anno: {bsonType: "int"},
      mese: {bsonType: "int", minimum: 1, maximum: 12},
      dipendente: {bsonType: "object"}, // snapshot anagrafico (embedded)
      totali: {bsonType: "object"}, voci: {bsonType: "array"},
      ratei: {bsonType: "array"}, contributi: {bsonType: "array"},
      addizionali: {bsonType: "array"}, giorni: {bsonType: "array"}
    }
  }}});
```

# Come si distribuiscono i dati

Su entrambi la chiave è `tenant_id`; cambia il *modo* in cui la distribuzione viene gestita.



Citus: **row-based** (scelto) vs  
schema-based

## Citus: distribuzione esplicita

**Row-based** per `tenant_id`; cataloghi **reference table**; tabelle di un tenant **co-locate** su un solo shard. Rebalance **manuale**, via **script dedicato**.

## MongoDB: distribuzione automatica

**Sharded cluster** (l'altra opzione era il **replica set**): shard key `tenant_id`, dati in **chunk** ridistribuiti da un **balancer** oltre una soglia ( $3 \times$  la range size). **Nei test si è ridotta la range size** per distribuire su tutti gli shard.

```
SELECT citus_add_node('10.0.1.5', 5432); -- sul coordinator
SELECT rebalance_table_shards(shard_transfer_mode := 'block_writes');
```



# Infrastruttura utilizzata per i test

---

Le misure sono state fatte su un cluster reale in cloud strutturato da una VM più potente che è stata utilizzata come coordinator ed entry point, e da quattro nodi-dati con le stesse caratteristiche hardware in modo da ottenere una scaling equo tra i nodi.

| Ruolo       | VM (IP privato)     | vCPU | RAM   | Funzione                                |
|-------------|---------------------|------|-------|-----------------------------------------|
| Coordinator | worker-1 (10.0.1.8) | 16   | 32 GB | Citus coordinator / Mongo config+mongos |
| Nodo-dati   | VM-1 (10.0.1.4)     | 4    | 16 GB | worker / shard                          |
| Nodo-dati   | VM-2 (10.0.1.5)     | 4    | 16 GB | worker / shard                          |
| Nodo-dati   | VM-3 (10.0.1.7)     | 4    | 16 GB | worker / shard                          |
| Nodo-dati   | VM-4 (10.0.1.6)     | 4    | 16 GB | worker / shard                          |

Ogni nodo ha un disco da 30 GB.

# Script di generazione dei dati: genera.py

---

Script **Python** parametrico e **riproducibile**: stesso comando  $\Rightarrow$  stesso dataset.

## Come sono generati i dati

- **Fonti ufficiali**: comuni con **codice catastale** (Belfiore), codici **ATECO**, **CCNL** e settori (export **CNEL**).
- **Dai cedolini reali** (Centropaghe): tipi voce, tipi contributo, causali di assenza.
- **Sintetici** (GDPR): ditte, dipendenti, cedolini; il **codice fiscale** è calcolato con l'algoritmo reale.

## Due formati, dagli stessi dati

- COPY (.sql)  $\rightarrow$  Citus
- JSONL  $\rightarrow$  mongoimport

**Caricamento** nei DB con gli script  
carica-citus.sh / carica-mongo.sh.

```
python3 data/genera.py --tenant-count 30 --dip-min 10 --dip-max 25 --seed 42 --out generato
bash scripts/carica-citus.sh generato/citus # oppure: carica-mongo.sh generato/mongo
```

# Metodologia di test

---

- ▶ **Dati:** lo **stesso dataset sintetico** sui due sistemi (seed fisso, riproducibile, calibrato sui cedolini reali), caricato in **bulk** (COPY / mongoimport) e cresciuto **a lotti**; suite speculare di 13 letture (dalle **single-shard** alle **cross-shard**) e 7 scritture.
- ▶ **Esecuzione:** **una query alla volta** sotto carico sostenuto, con pgbench (Citus) e un loop mongosh (Mongo); letture a **parametri randomizzati**, scritture a **tenant fisso**; cache **fredda** (scaling nodi) o **calda** (scaling dati).
- ▶ **Raccolta e analisi:** contatori nativi per-nodo con **azzera → run → delta** (pg\_stat\_statements via run\_command\_on\_all\_nodes, serverStatus, docker stats, /proc). Ripetizioni **per tipo di test:** letture **media su 3 run**, scritture **una sola esecuzione**, guasti **10 run** (ogni volta con un nodo random spento). I CSV diventano poi tabelle e grafici.

# Gli script, per ogni tipo di test

Ogni esperimento è automatizzato da script dedicati, **speculari** tra i due sistemi.

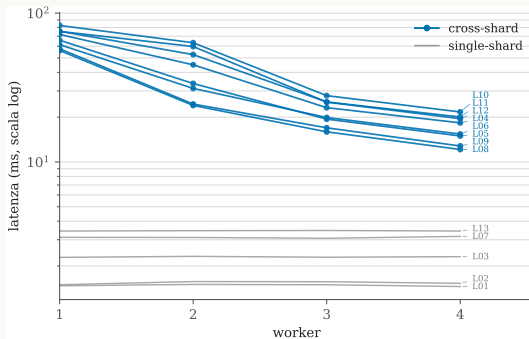
| Esperimento      | PostgreSQL + Citus                      | MongoDB                                |
|------------------|-----------------------------------------|----------------------------------------|
| Caricamento dati | carica-citus.sh (COPY)                  | carica-mongo.sh (mongoimport)          |
| Suite letture    | esegui-suite-citus.sh                   | esegui-suite-mongo.sh                  |
| Scaling per nodi | citus-add-node.sh<br>citus-rebalance.sh | scala-nodi-mongo.sh                    |
| Scaling per dati | scala-dati-citus.sh                     | scala-dati-mongo.sh                    |
| Scritture        | esegui-scritture-citus.sh               | esegui-scritture-mongo.sh              |
| Guasti           | test-guasto-citus.sh                    | test-guasto-mongo.sh                   |
| Cache / media    | azzer-cache-citus.sh<br>media-citus.sh  | azzer-cache-mongo.sh<br>media-mongo.sh |

I CSV raccolti diventano poi i grafici del capitolo risultati (`grafici.py`, `matplotlib`).

# Scalabilità sui nodi: PostgreSQL + Citus

A parità di dati si passa da **1 a 4 nodi**, con **rebalance esplicito** a ogni passo.

- ▶ Le analitiche **cross-shard** scalano **quasi linearmente**: speedup **3.8–4.6×** con 4 nodi (L04 72→18 ms, L11 76→20 ms).
- ▶ Le **single-shard** restano piatte ( $\sim 1.5$  ms): colpiscono un solo shard.
- ▶ Il lavoro *per worker* (CPU, righe elaborate) cala coi nodi.

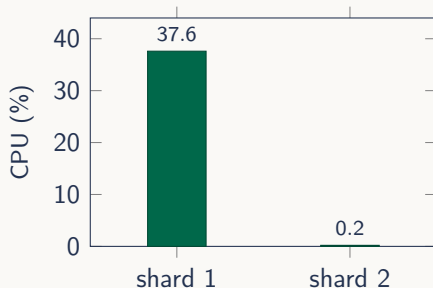


# Scalabilità sui nodi: MongoDB

A parità di dati si aggiungono shard, ma **il balancer non distribuisce a questa scala**: la collezione cedolini (~42 MB) resta su un solo shard, perché la soglia di migrazione ( $3 \times$  la range size, cioè 384 MB) non è mai raggiunta. Anche riducendo la range size e splittando i chunk a mano, in **888 round** il balancer **non migra** i chunk sui nodi.

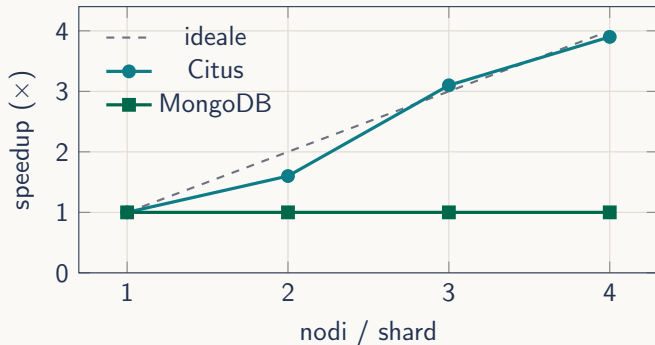
## Conseguenza

A 1 e 2 shard la latenza è **identica** (L04 7→7 ms, L11 12→12 ms): aggiungere shard **non porta beneficio**. Mongo resta però più **veloce in assoluto**.



CPU per shard sotto carico (2 shard): il 2° resta inattivo

## Scalabilità sui nodi: confronto in sintesi



Speedup su L04, cioè la latenza a 1 nodo divisa per quella a  $n$  nodi. Citus si avvicina allo scaling **ideale** ( $\sim 4\times$  con 4 nodi), mentre MongoDB resta a  $1\times$ : a questa scala il balancer non ridistribuisce i dati.

## Scalabilità sui nodi: tutte le query in lettura (1–4 nodi)

| Query — latenza (ms)    | Citux (nodi) |      |      |      | sp.  | MongoDB (shard) |      |      |
|-------------------------|--------------|------|------|------|------|-----------------|------|------|
|                         | 1            | 2    | 3    | 4    | 1→4  | 1               | 2    | 3    |
| L01 cedolino singolo    | 1.5          | 1.5  | 1.5  | 1.5  | 1.0× | 2.3             | 2.3  | 2.3  |
| L02 cedolini ditta      | 1.5          | 1.6  | 1.6  | 1.5  | 1.0× | 2.7             | 2.7  | 2.7  |
| L03 costo per centro    | 2.3          | 2.3  | 2.3  | 2.3  | 1.0× | 3.1             | 3.1  | 3.1  |
| L04 gender gap mediano  | 71.9         | 45.0 | 23.2 | 18.3 | 3.9× | 7.2             | 7.2  | 7.1  |
| L05 top-N retribuzioni  | 65.8         | 33.7 | 19.4 | 14.9 | 4.4× | 3.7             | 3.7  | 3.7  |
| L06 assenteismo/mese    | 61.4         | 31.2 | 19.9 | 15.4 | 4.0× | 100             | 99.5 | 99.9 |
| L07 straordinari/centro | 3.1          | 3.1  | 3.1  | 3.2  | 1.0× | 2.9             | 2.9  | 2.9  |
| L08 trend costo lavoro  | 56.1         | 24.0 | 15.9 | 12.1 | 4.6× | 13.7            | 10.3 | 10.3 |
| L09 organico/livello    | 57.7         | 24.5 | 17.0 | 12.8 | 4.5× | 6.5             | 6.5  | 6.6  |
| L10 cedolino completo   | 82.7         | 63.3 | 27.9 | 21.7 | 3.8× | 2.3             | 2.3  | 2.3  |
| L11 spesa per categoria | 75.6         | 52.7 | 25.3 | 20.1 | 3.8× | 12.4            | 12.5 | 12.6 |
| L12 ferie residue       | 75.4         | 59.7 | 25.2 | 19.5 | 3.9× | 141             | 142  | 142  |
| L13 riepilogo assenze   | 3.4          | 3.5  | 3.5  | 3.4  | 1.0× | 11.0            | 11.0 | 11.0 |

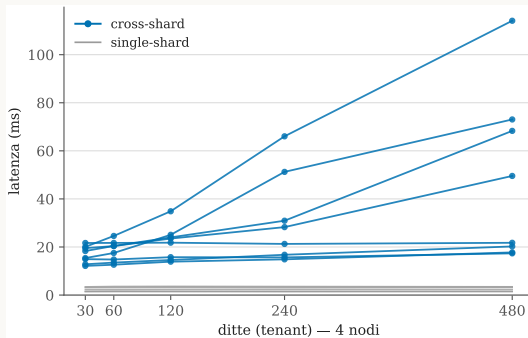
All'aumentare dei nodi le analitiche **cross-shard** di Citux migliorano **quasi linearmente** (fino a  $\sim 4\times$ ), mentre le **single-shard** restano costanti. In MongoDB la latenza **non cambia** aggiungendo shard (il balancer non ridistribuisce a questa scala), pur restando più rapida in assoluto.



# Scalabilità sui dati: PostgreSQL + Citus

Mantenendo **4 nodi fissi** e aumentando il volume dei dati, in particolare aumentando il numero di ditte da 30 a 480 ditte.

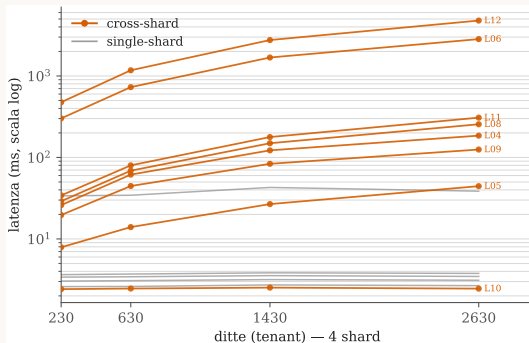
- ▶ Le **cross-shard** rallentano al crescere dei dati, con un degrado che **accelera** (L11 20→114 ms, L06 15→73, L04 18→68): più righe da scandire per shard.
- ▶ Le **single-shard** restano **piatte** (~1.5 ms): insensibili al volume globale.
- ▶ `blk_read=0`: i dati stanno ancora in RAM, quindi il degrado è **CPU-bound**.



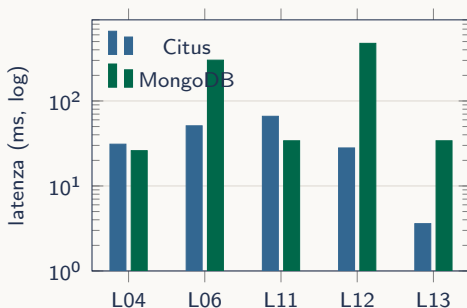
# Scalabilità sui dati: MongoDB

A 4 shard fissi si è spinto il volume **ben oltre le 480 ditte di Citus**, fino a **2630 ditte** (~48.000 dipendenti).

- ▶ A questa scala, e con la **range size ridotta** sotto il default, il balancer ha **distribuito** su **tutti e 4 gli shard** (CPU 48–65% ciascuno).
- ▶ Le **cross-shard** crescono col volume (L04 26→185 ms, L11 34→308); L06 e L12 arrivano ai **secondi** (~2.8 e 4.8 s).
- ▶ Le query mirate restano **piatte** (~2.6 ms).



# Scalabilità sui dati: dove si vedono le differenze



Latenza a pari scala (~230–240 ditte, 4 nodi)

| Query (ms)         | Citus | Mongo |           |
|--------------------|-------|-------|-----------|
| L06 assenteismo    | 51    | 302   | Citus 6×  |
| L12 ferie residue  | 28    | 476   | Citus 17× |
| L13 riepilogo ass. | 3.6   | 34    | Citus 9×  |
| L04 gender gap     | 31    | 26    | ≈ pari    |
| L11 spesa/cat.     | 66    | 34    | Mongo 2×  |

Citus domina le query **multi-entità** (assenze, ratei: **join indicizzati**); MongoDB vince dove l'**embedding** serve già l'aggregato.

# Scalabilità sui dati: Citus vs MongoDB (latenza, ms)

| Query              | Citus (ditte) |      |      |      |       | MongoDB (ditte) |      |      |      | vince     |
|--------------------|---------------|------|------|------|-------|-----------------|------|------|------|-----------|
|                    | 30            | 60   | 120  | 240  | 480   | 230             | 630  | 1430 | 2630 |           |
| L01 cedolino       | 1.5           | 1.5  | 1.5  | 1.4  | 1.6   | 2.6             | 2.6  | 2.7  | 2.7  | Citus     |
| L02 cedolini ditta | 1.5           | 1.5  | 1.5  | 1.6  | 1.5   | 3.0             | 3.1  | 3.2  | 3.1  | Citus     |
| L03 costo/centro   | 2.3           | 2.3  | 2.4  | 2.4  | 2.4   | 3.4             | 3.4  | 3.5  | 3.5  | ≈         |
| L04 gender gap     | 18.3          | 20.4 | 24.0 | 30.9 | 68.3  | 26.0            | 61.6 | 122  | 185  | Mongo     |
| L05 top-N          | 14.9          | 14.8 | 15.8 | 15.7 | 17.4  | 7.9             | 14.0 | 26.8 | 44.4 | Mongo 2×  |
| L06 assenteismo    | 15.4          | 17.5 | 25.1 | 51.2 | 73.1  | 302             | 729  | 1685 | 2834 | Citus 6×  |
| L07 straordinari   | 3.2           | 3.2  | 3.2  | 3.2  | 3.3   | 3.6             | 3.7  | 3.8  | 3.8  | ≈         |
| L08 trend costo    | 12.1          | 12.6 | 13.9 | 14.9 | 17.8  | 29.0            | 68.8 | 149  | 255  | Citus 2×  |
| L09 organico       | 12.8          | 13.5 | 14.6 | 16.8 | 20.2  | 19.7            | 44.6 | 83.6 | 125  | ≈         |
| L10 ced. completo  | 21.7          | 21.7 | 21.8 | 21.3 | 21.7  | 2.4             | 2.5  | 2.5  | 2.5  | Mongo 9×  |
| L11 spesa/cat.     | 20.1          | 24.6 | 34.9 | 66.0 | 114.1 | 34.2            | 80.0 | 178  | 308  | Mongo 2×  |
| L12 ferie residue  | 19.5          | 20.4 | 23.5 | 28.3 | 49.6  | 476             | 1174 | 2760 | 4786 | Citus 17× |
| L13 riepilogo      | 3.4           | 3.6  | 3.6  | 3.6  | 3.5   | 33.8            | 34.4 | 42.8 | 38.6 | Citus 9×  |

**Vince** = più veloce a pari scala (Citus 240 vs Mongo 230 ditte). Citus domina le **query multi-entità** (assenze, ratei: L06, L12, L13); MongoDB l'**aggregato** (cedolino completo L10) e i percentili/top-N (L04, L05, L11).

# Scalabilità sui dati: Citus, tutte le metriche (30–480 ditte)

| Query            | latenza (ms) |      |      |      |       | throughput (op/s) |     |     |     |     | CPU worker (%) |    |     |     |     | exec worker (ms) |      |      |      |      | righe/worker |        |        |        |        |
|------------------|--------------|------|------|------|-------|-------------------|-----|-----|-----|-----|----------------|----|-----|-----|-----|------------------|------|------|------|------|--------------|--------|--------|--------|--------|
|                  | 30           | 60   | 120  | 240  | 480   | 30                | 60  | 120 | 240 | 480 | 30             | 60 | 120 | 240 | 480 | 30               | 60   | 120  | 240  | 480  | 30           | 60     | 120    | 240    | 480    |
| L01 cedolino     | 1.5          | 1.5  | 1.5  | 1.4  | 1.6   | 688               | 659 | 655 | 705 | 643 | 3              | 6  | 6   | 5   | 4   | 34               | 36   | 36   | 38   | 34   | 1718         | 1647   | 1636   | 1762   | 1607   |
| L02 ced. ditta   | 1.5          | 1.5  | 1.5  | 1.6  | 1.5   | 654               | 659 | 674 | 635 | 682 | 4              | 5  | 4   | 4   | 4   | 43               | 44   | 45   | 33   | 36   | 1636         | 1648   | 1684   | 1588   | 1706   |
| L03 costo/centro | 2.3          | 2.3  | 2.4  | 2.4  | 2.4   | 433               | 438 | 424 | 414 | 423 | 7              | 7  | 7   | 8   | 8   | 161              | 164  | 171  | 175  | 185  | 51988        | 52548  | 50820  | 49680  | 50748  |
| L04 gender gap   | 18.3         | 20.4 | 24.0 | 30.9 | 68.3  | 55                | 49  | 42  | 32  | 15  | 24             | 26 | 26  | 28  | 38  | 347              | 623  | 947  | 1430 | 1325 | 75667        | 164762 | 283664 | 454734 | 411049 |
| L05 top-N        | 14.9         | 14.8 | 15.8 | 15.7 | 17.4  | 67                | 68  | 63  | 64  | 58  | 20             | 22 | 23  | 27  | 30  | 266              | 423  | 587  | 953  | 1475 | 60240        | 89401  | 98425  | 102080 | 92320  |
| L06 assenteismo  | 15.4         | 17.5 | 25.1 | 51.2 | 73.1  | 65                | 57  | 40  | 20  | 14  | 17             | 32 | 39  | 68  | 91  | 923              | 2445 | 3337 | 3655 | 5274 | 187125       | 326326 | 262342 | 134554 | 94256  |
| L07 straordinari | 3.2          | 3.2  | 3.2  | 3.2  | 3.3   | 317               | 317 | 309 | 312 | 307 | 11             | 11 | 11  | 11  | 12  | 441              | 478  | 474  | 483  | 480  | 3165         | 3173   | 3093   | 3122   | 3068   |
| L08 trend costo  | 12.1         | 12.6 | 13.9 | 14.9 | 17.8  | 82                | 79  | 72  | 67  | 56  | 12             | 14 | 18  | 27  | 39  | 351              | 673  | 1148 | 2091 | 3460 | 49500        | 68904  | 66960  | 64608  | 54144  |
| L09 organico     | 12.8         | 13.5 | 14.6 | 16.8 | 20.2  | 78                | 74  | 69  | 60  | 49  | 13             | 13 | 14  | 15  | 17  | 125              | 210  | 336  | 546  | 881  | 58234        | 104710 | 185220 | 321186 | 535095 |
| L10 ced. compl.  | 21.7         | 21.7 | 21.8 | 21.3 | 21.7  | 46                | 46  | 46  | 47  | 46  | 31             | 32 | 31  | 31  | 30  | 112              | 115  | 115  | 119  | 116  | 115          | 115    | 115    | 118    | 115    |
| L11 spesa/cat.   | 20.1         | 24.6 | 34.9 | 66.0 | 114.1 | 50                | 41  | 29  | 15  | 9   | 25             | 27 | 29  | 42  | 52  | 543              | 1102 | 1292 | 1404 | 1718 | 69158        | 136854 | 195912 | 213332 | 246070 |
| L12 ferie res.   | 19.5         | 20.4 | 23.5 | 28.3 | 49.6  | 51                | 49  | 43  | 35  | 20  | 30             | 35 | 38  | 44  | 65  | 528              | 1007 | 1576 | 2569 | 3100 | 3842         | 7365   | 12780  | 21240  | 24240  |
| L13 riepilogo    | 3.4          | 3.6  | 3.6  | 3.6  | 3.5   | 291               | 279 | 276 | 280 | 289 | 10             | 10 | 10  | 10  | 10  | 230              | 233  | 224  | 228  | 193  | 7281         | 6978   | 6895   | 7010   | 7220   |

blk\_read=0 (dati in RAM): il degrado è CPU-bound. Sulle cross-shard le righe scandite per worker crescono col volume e il throughput cala; le single-shard restano costanti.

# Scalabilità sui dati: MongoDB, tutte le metriche

| Query              | latenza (ms) |      |      |      | throughput (op/s) |     |      |      | CPU media/shard (%) |     |      |      |
|--------------------|--------------|------|------|------|-------------------|-----|------|------|---------------------|-----|------|------|
|                    | 230          | 630  | 1430 | 2630 | 230               | 630 | 1430 | 2630 | 230                 | 630 | 1430 | 2630 |
| L01 cedolino       | 2.6          | 2.6  | 2.7  | 2.7  | 385               | 383 | 367  | 374  | 6                   | 5   | 5    | 6    |
| L02 cedolini ditta | 3.0          | 3.1  | 3.2  | 3.1  | 329               | 325 | 316  | 321  | 6                   | 6   | 6    | 6    |
| L03 costo/centro   | 3.4          | 3.4  | 3.5  | 3.5  | 294               | 291 | 282  | 289  | 6                   | 7   | 6    | 6    |
| L04 gender gap     | 26.0         | 61.6 | 122  | 185  | 38                | 16  | 8    | 5    | 14                  | 15  | 18   | 22   |
| L05 top-N          | 7.9          | 14.0 | 26.8 | 44.4 | 127               | 72  | 37   | 22   | 31                  | 43  | 54   | 58   |
| L06 assenteismo    | 302          | 729  | 1685 | 2834 | 3                 | 1   | 1    | 0    | 59                  | 66  | 70   | 75   |
| L07 straordinari   | 3.6          | 3.7  | 3.8  | 3.8  | 274               | 270 | 261  | 265  | 8                   | 8   | 8    | 8    |
| L08 trend costo    | 29.0         | 68.8 | 149  | 256  | 34                | 14  | 7    | 4    | 49                  | 58  | 64   | 69   |
| L09 organico       | 19.7         | 44.6 | 83.6 | 125  | 51                | 22  | 12   | 8    | 14                  | 16  | 20   | 23   |
| L10 ced. completo  | 2.4          | 2.5  | 2.5  | 2.5  | 413               | 406 | 396  | 407  | 4                   | 3   | 3    | 3    |
| L11 spesa/cat.     | 34.2         | 80.0 | 178  | 308  | 29                | 12  | 6    | 3    | 48                  | 56  | 61   | 65   |
| L12 ferie residue  | 476          | 1174 | 2760 | 4786 | 2                 | 1   | 0    | 0    | 57                  | 62  | 68   | 89   |
| L13 riepilogo      | 33.8         | 34.4 | 42.8 | 38.6 | 30                | 29  | 23   | 26   | 17                  | 17  | 17   | 21   |

Disco letto  $\approx 0$  (dati in RAM). Le cross-shard (L06, L12) **saturano la CPU su tutti gli shard** e crollano nel throughput; le mirate (L01, L10) restano  $\sim 400$  op/s.

# Confronto nelle letture: comportamento e idoneità

## PostgreSQL + Citus

**Sui nodi:** **scala** — col rebalance esplicito le analitiche cross-shard migliorano  $\sim 4\times$  da 1 a 4 nodi.

**Sui dati:** degrada col volume (il degrado *accelera*), ma le query mirate restano costanti.

**Più adatto a:** analitiche **multi-entità** — assenze L06 ( $6\times$ ), ferie/ratei L12 ( $17\times$ ), riepiloghi L13 ( $9\times$ ): *join* indicizzati.

## MongoDB

**Sui nodi:** col dataset piccolo del test **non ridistribuisce**; più veloce in assoluto.

**Sui dati:** a volume grande distribuisce (chunk ridotti, fino a 2630 ditte); scandire gli **array annidati** (L06, L12) porta ai *secondi*.

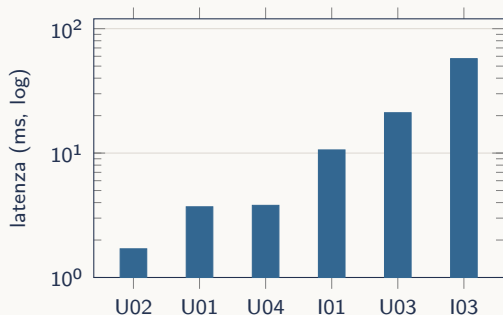
**Più adatto a:** informazioni **aggregate** — cedolino completo L10 ( $9\times$ , un solo findOne) e aggregati/percentili semplici (L04, L05, L11).

## In sintesi — caso d'uso HR

Nessuno dei due prevale in assoluto: l'idoneità **dipende dal tipo di query**. Per il **reporting analitico multi-entità** **Citus risulta più efficiente e scala aggiungendo nodi**; per le **letture del singolo cedolino** (l'aggregato) **MongoDB è più rapido**.

# Scritture: PostgreSQL + Citus

- **Co-localizzazione:** ogni scrittura di un tenant colpisce **un solo nodo**.
- Il costo cresce con la **complessità transazionale**: dall'UPDATE di 1 riga (U02, 1.7 ms) alla **transazione multi-tabella** I03 (57 ms) — fattore  $\sim 35\times$ .
- L'unica che tocca **tutti i nodi** è U03 (reference table replicata): 21 ms, ma resta contenuta.

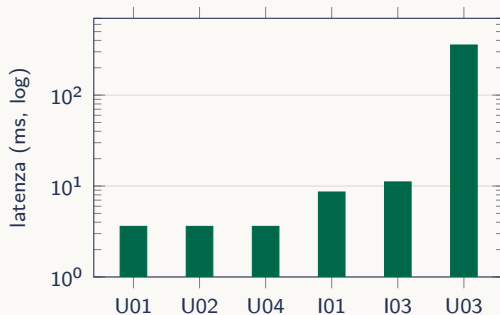


Latenza per operazione (4 nodi,  $N=300$ , tenant fisso)



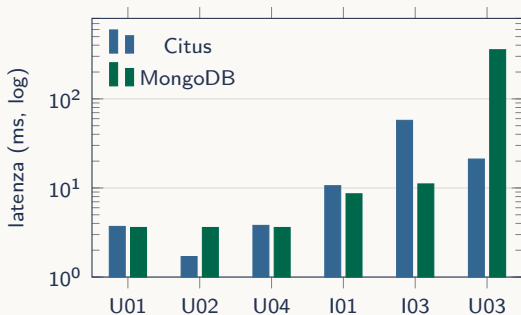
# Scrittura: MongoDB

- Inserire un **cedolino** (I03) è **economico**: 11 ms, **un solo documento**, niente transazione multi-tabella.
- Le scritture singole sono veloci ( $\sim 3.6$  ms).
- **Ma** aggiornare un **dato condiviso** (U03) costa **356 ms**: lo snapshot denormalizzato va riscritto in **ogni cedolino**  $\Rightarrow$  CPU  $\sim 95\%$ , 265 MB scritti.



U03 esplode: *write amplification* sull'embedding

# Scritture: confronto Citus vs MongoDB



**I03** cedolino: **Mongo 5×** — 1 documento vs transazione su 7 tabelle.

**U03** dato condiviso: **Citus 17×** — 1 riga normalizzata vs snapshot in ogni cedolino.

Le altre scritture: **sostanzialmente pari**.

**Scalabilità** (entrambi): ogni scrittura **co-locata su un nodo** — aggiungere nodi non abbassa la singola scrittura, ma il **throughput** cresce con i tenant in parallelo.

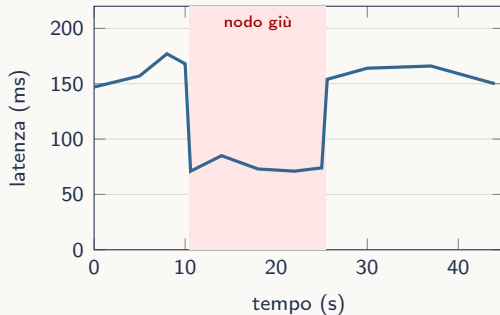
## In sintesi

L'**embedding** rende economico *scrivere l'aggregato* ma costoso *propagare una modifica condivisa*; la **normalizzazione** fa l'opposto.

# Guasti: PostgreSQL + Citus

**Test:** L04 in loop per 45 s, un worker spento tra 10 e 25 s.

- ▶ Col nodo del tenant giù la query **fallisce all'istante** (**fail-fast**, connection refused).
- ▶ **righe** → **0**: nessun risultato parziale (CP).
- ▶ Tanti errori «brevi» ( $\sim 70$  ms)  $\Rightarrow$  error-rate **alto** ( **$\sim 51\%$** ).
- ▶ Recupero **automatico** al ritorno del nodo ( $\sim 0.5$  s).

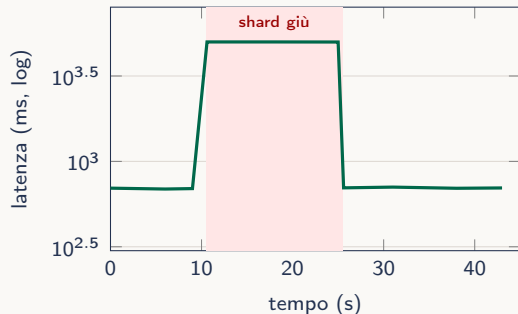


Fallimento **immediato**: la latenza *cala* (errori istantanei)

# Guasti: MongoDB

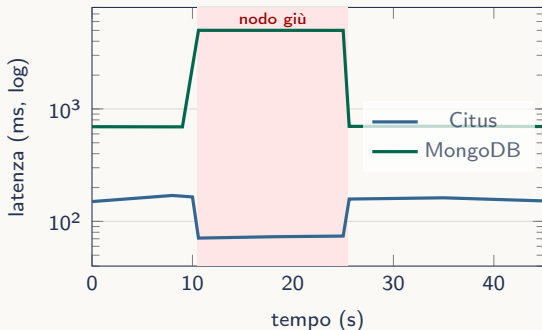
**Test:** stesso protocollo, shard con i dati spento tra 10 e 25 s.

- ▶ Con lo shard giù la query si **blocca fino al timeout** ( $\sim 5$  s per operazione).
- ▶ **righe**  $\rightarrow$  **0**: nessun risultato parziale (CP).
- ▶ Pochi errori «lunghi»  $\Rightarrow$  error-rate **basso** ( $\sim 7\%$ ), ma ognuno costa 5 s.
- ▶ Recupero **automatico** al ritorno dello shard ( $\sim 0.25$  s).



Blocco: la latenza **sale a  $\sim 5$  s** (timeout per operazione)

# Guasti: confronto Citus vs MongoDB



Durante il down, per **entrambi** righe  $\rightarrow 0$  (nessun parziale)

**Elementi comuni** (entrambi CP):

indisponibilità reale  $\sim 15$  s, assenza di risultati parziali (righe = 0), ripristino automatico.

**Modalità di fallimento differente:** **Citus** rifiuta la connessione all'istante ( $\sim 70$  ms per tentativo); **MongoDB** attende la scadenza del timeout ( $\sim 5$  s per operazione).

## Lettura dell'error-rate

Il valore grezzo (51% contro 7%) non è confrontabile: dipende dalla **durata di ogni tentativo fallito**, non dalla gravità del guasto. I fallimenti *fail-fast*, istantanei, si ripetono molte più volte nella stessa finestra; il riferimento corretto è l'**indisponibilità reale**, identica per entrambi.

# Evoluzione dello schema in corso d'opera

Scenario: aggiungere **turni e ferie** a un sistema **già popolato**.

|                 | PostgreSQL + Citus<br>schema esplicito                          | MongoDB<br>schema flessibile                             |
|-----------------|-----------------------------------------------------------------|----------------------------------------------------------|
| Come si applica | ALTER TABLE o nuova tabella distribuita, co-locata su tenant_id | Si scrivono i nuovi campi/array solo nei documenti nuovi |
| Dati esistenti  | <b>Backfill</b> e propagazione a <b>tutti gli shard</b>         | <b>Intatti</b> : documenti vecchi e nuovi coesistono     |
| Governo         | Vincoli e integrità <b>garantiti dal DBMS</b>                   | Demandato all'applicazione ( <i>schema-on-read</i> )     |
| Costo / rischio | Migrazione transazionale, possibili <b>lock</b>                 | Estensione <b>quasi immediata</b>                        |

## In sintesi

Il **documentale** privilegia l'**agilità evolutiva** (i dati esistenti non si toccano); il **relazionale** le **garanzie di integrità** durante il cambiamento.

# Conclusioni

Gli esperimenti **confermano l'ipotesi iniziale**: in **lettura e scrittura**, le operazioni su **più entità** sono più efficienti sul modello **relazionale** (Citus: L06, L12, L13, U03), quelle sull'**aggregato o singola entità** sul **non relazionale** (MongoDB: L10, I03). Sulla **scalabilità in ambiente distribuito** i due sistemi seguono approcci distinti:

## PostgreSQL + Citus

Distribuzione **esplicita** (rebalance on-demand), a qualsiasi volume: aggiungendo nodi le analitiche migliorano  $3.8-4.6\times$  (1→4 nodi).

## MongoDB

Distribuzione **automatica, legata al volume**: col dataset piccolo del test resta su un solo shard; **a volume pieno** distribuisce su tutti gli shard.

## In sintesi

**Citus** ridistribuisce su richiesta, con guadagno immediato a qualsiasi volume ( $\sim 4\times$ ); **MongoDB** usa un balancer automatico, che interviene solo oltre una soglia di dati (raggiunta col dataset finale, non con quello iniziale).